

Pipeline na prática: Actions + Docker

Processos de Software – Sprint 2 (prática)

Prof. Fernando · UFRN · 2026.1

De onde paramos

Aula passada: **teoria** de DevOps.

Vimos	Hoje
CALMS, 3 caminhos, DORA	Mão na massa
"CI é integrar diariamente"	Escrever um <code>ci.yml</code> que roda
"Docker garante reprodutibilidade"	Escrever um <code>Dockerfile</code> que faz o <i>build</i>
"Pipeline como filtro"	Ver o filtro funcionando (e quebrando)

Objetivo de hoje: **se o lint passa e os testes passam no CI, está tudo pronto para a Sprint 2.**

Plano da aula

1. Anatomia de um pipeline GitHub Actions ~10 min
2. Dockerfile multi-stage: por que e como ~10 min
3. **Prática guiada no Codespaces** ~30 min
 - Etapa 1: workflow de CI
 - Etapa 2: Dockerfile
 - Etapa 3: docker-compose
4. Dicas para o vídeo de 8min ~5 min

GitHub Actions: vocabulário mínimo

Termo	O que é
Workflow	Arquivo YAML em <code>.github/workflows/</code> . Um pipeline.
Event	O que dispara o workflow (<code>push</code> , <code>pull_request</code> , ...)
Job	Conjunto de passos que rodam numa mesma máquina virtual
Step	Um comando ou uso de uma <code>action</code> reutilizável
Runner	A VM que executa o job (<code>ubuntu-latest</code> é o padrão)
Action	Bloco reutilizável (ex: <code>actions/checkout@v4</code>)

Vamos escrever um arquivo YAML que descreve uma sequência de comandos que rodam numa VM Ubuntu temporária no GitHub.

Anatomia: o `ci.yml` mais simples possível

```
name: CI                                # nome que aparece na aba Actions

on:                                     # quando rodar
  push:
    branches: [main]
  pull_request:

jobs:
  test:                                # nome do job (livre)
    runs-on: ubuntu-latest             # qual VM
    steps:
      - uses: actions/checkout@v4      # baixa o código do repo
      - uses: actions/setup-python@v5  # instala Python
        with:
          python-version: '3.12'
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: pytest                     # roda os testes
```

É isso. Cinco passos lógicos: dispara → VM sobe → baixa código → instala stack → roda teste.

0 que torna um pipeline "bom"

Não é complexidade. É **cobertura dos pontos de falha**:

Etapa	0 que ela pega
<code>pip install</code>	Dependência quebrada ou conflito
<code>ruff check</code>	Lint: imports não usados, código morto, estilo
<code>pytest</code>	Comportamento quebrado
<code>pytest --cov</code>	Código sem cobertura de teste
<code>mypy</code>	Erros de tipo (opcional, para projetos tipados)
<code>docker build</code>	Imagem não builda

Cada etapa é um **filtro**. Falhou em qualquer uma → não passa adiante.

Cache: do "lento" ao "rápido"

Sem cache: cada build baixa todas as dependências de novo. Pode levar 1-2 min.

```
- uses: actions/setup-python@v5
  with:
    python-version: '3.12'
    cache: 'pip'                # ← uma linha mais.
```

Resultado típico: build cai de 90s para 20s.

Lead time é uma das métricas DORA. Cache no CI **mexe diretamente** nessa métrica.

Dockerfile: por que multi-stage?

Dockerfile ingênuo (NÃO façam isso):

```
FROM python:3.12
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "-m", "app.main"]
```

Problemas:

- Imagem final ~1 GB (Python "completo" + pip cache + ferramentas de build)
- Roda como `root` (sem segurança em camadas)
- Copia `tests/`, `.git/`, etc. para produção
- Reinstala tudo a cada mudança no código

Dockerfile multi-stage

```
# Stage 1: builder – instala dependências
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Stage 2: runtime – só o necessário
FROM python:3.12-slim
RUN useradd --create-home app
USER app
WORKDIR /home/app
COPY --from=builder --chown=app:app /root/.local /home/app/.local
COPY --chown=app:app app/ ./app/
ENV PATH=/home/app/.local/bin:$PATH
CMD ["python", "-m", "app.main"]
```

Imagem final: **~150 MB**, rodando como usuário `app`.

Lean aplicado ao CI: **eliminar desperdício** otimizando e provendo mais segurança no processo de automação de build.

Por que `--user` e copiar `/root/.local`?

```
RUN pip install --user --no-cache-dir -r requirements.txt
# ...
COPY --from=builder --chown=app:app /root/.local /home/app/.local
```

- `--user`: instala em `~/.local` em vez de site-packages global. Tudo num lugar só.
- `--no-cache-dir`: além de instalar em `~/.local`, o pip guarda uma cópia dos wheels em `~/.cache/pip` para acelerar futuras instalações. Numa imagem Docker isso nunca é usado (o `pip install` só roda uma vez no build), então o cache fica ocupando espaço à toa. `--no-cache-dir` desliga essa cópia. Tipicamente economiza 30-50MB.
- A stage final copia **apenas** `~/.local` — sem pip, sem ferramentas de build, sem cache.

A stage de build pode ter o que for preciso. A stage de runtime tem **só o mínimo para rodar**.

E rodar como root? Por que não?

```
RUN useradd --create-home app  
USER app
```

Defesa em profundidade: se a aplicação for comprometida (RCE via lib vulnerável, p.ex.), o atacante fica preso num usuário sem privilégios.

Sem isso: o atacante **é root** dentro do container — pode escrever em qualquer lugar, instalar ferramentas, escalar privilégios.

É uma linha de proteção. Não é tudo, mas é gratuito e funciona.

docker-compose: orquestrar serviços

Sua API quase nunca roda sozinha. Tem banco, cache, fila...

```
services:
  api:
    build: .
    ports: ["8080:8080"]
    environment:
      DATABASE_URL: postgres://app:app@db:5432/app
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: app
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s
```

Um comando: `docker compose up` . Um comando para derrubar tudo: `docker compose down` .

Por que `healthcheck` + `depends_on` ?

Sem healthcheck:

```
api sobe → tenta conectar no banco → banco ainda inicializando → CRASH
```

Com healthcheck:

```
db sobe → healthcheck "pg_isready" passa →  
→ api sobe → conecta com sucesso
```

`depends_on` simples só espera o **container** subir. Com `condition: service_healthy`, espera o **serviço** estar pronto. Diferença sutil, mas é o que separa "funciona às vezes" de "funciona sempre".

Onde Codespaces entra

GitHub Codespaces = **VM Ubuntu pronta no navegador**, com Docker-in-Docker.

Vocês não precisam	Codespaces dá
Instalar Docker no PC do lab	Já vem
Permissão de admin	Roda no GitHub
Stack Python local	<code>devcontainer</code> configura

Atenção: a cota gratuita do Codespaces é generosa, mas finita. Parem o codespace quando não estiverem usando: aba Codespaces → ... → **Stop codespace**.

Prática agora – 3 etapas

Repo base: *(link enviado no Discord)* — forkar para a conta do time.

Etapa	Tempo	Resultado
1 — Workflow de CI	10 min	Pipeline rodando, badge verde
2 — Dockerfile multi-stage	10 min	Imagem buildando local e no CI
3 — docker-compose	10 min	App + Postgres rodando juntos

Cada etapa tem um arquivo `STEP-N.md` no repo com as instruções.

Vamos quebrar coisas de propósito. É a melhor forma de entender o que cada filtro pega.

Sobre o vídeo de 8min

Vocês vão **gravar a tela** mostrando a Sprint 2. Hoje vocês geram a matéria-prima.

Roteiro sugerido (8 min):

Parte	Tempo	Conteúdo
Pipeline em ação	2 min	Push → CI roda → verde
Quebrar de propósito	1 min	Push com teste falhando → CI vermelho
Docker rodando	2 min	<code>docker compose up</code> , app responde
Métricas DORA	2 min	Dois números + interpretação
Reflexão	1 min	O que mudou no fluxo do time?

Não narrem o YAML linha por linha. Mostrem o **efeito**. O artefato fala por si.

Métricas DORA: como coletar (mínimo viável)

Vocês não precisam de dashboard. Precisam de **dois números** pelo menos (podem coletar mais se conseguirem e no trabalho isso será convertido em um pequeno bônus).

Frequência de deploy (*proxy: merges em main*):

```
git log --since="2 weeks ago" --oneline main | wc -l
```

Lead time (*commit → merge em main*):

- Anotem na descrição do PR: data do primeiro commit e data do merge.
- Calculem a média.

Não *mascarar* um resultado só porque ele seja aparentemente **ruim**. Peguem o número **real** e elaborem em grupo o que será feito para melhorar. Isso é **DORA**, e tem relação com a cultura de **melhoria contínua** que vimos no Lean e no Scrum.

Checklist antes de ir embora

Sprint 2 entrega prazo: 19/05 23:59 :

- ☐ `.github/workflows/ci.yml` rodando verde
- ☐ `Dockerfile` multi-stage compila
- ☐ `docker-compose.yml` sobe app + dependência
- ☐ `METRICS.md` com 2 métricas DORA reais
- ☐ Retrospectiva da Sprint 2 documentada
- ☐ Vídeo 8 min gravado e linkado

Se até o fim da prática de hoje vocês tiverem os três primeiros, **a sprint está 70% feita.**

Referências da aula

- GitHub Actions docs — docs.github.com/actions
- Docker multi-stage builds — docs.docker.com/build/building/multi-stage
- Compose specification — compose-spec.io
- DORA — *Four Keys* — dora.dev/guides/dora-metrics-four-keys
- Codespaces docs — docs.github.com/codespaces